String processing

CSC103 Programming Fundamentals / Lecture 13

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

For a nonempty email-like string with one @, extract the username and domain. Test ali@campus.edu.

Find the @ position and reject missing or empty parts.

Learning objectives

- Use common String methods
- Compare text content and extract substrings
- Convert numeric text and numeric values

CLO-2
Length and character access
Content comparison
Substrings and searching
Numbers and text

Length and character access

- length() counts UTF-16 code units.
- charAt(index) uses positions from 0 to length()-1.
- Basic Latin examples let each char correspond to one visible character.

Example

```
String s = "Java";
s.length()    // 4
s.charAt(0)   // J
```

Content comparison

- equals compares String content.
- equalsIgnoreCase ignores case distinctions for its comparison.
- compareTo returns a negative value, zero or a positive value.

Example

```
"cat".equals("cat")         // true
"Cat".equals("cat")         // false
"cat".compareTo("dog") < 0 // true
```

Substrings and searching

- substring(begin,end) includes begin and excludes end.
- indexOf returns the first matching position or -1.
- trim returns a result without leading or trailing basic whitespace.

Example

```
String code = "CS-103";
code.substring(3)     // "103"
code.substring(0, 2)  // "CS"
code.indexOf("-")     // 2
```

Numbers and text

- Integer.parseInt and Double.parseDouble parse numeric text.
- String.valueOf converts a value to text.
- Malformed numeric text raises NumberFormatException.

Example

```
int n = Integer.parseInt("42");
String label = String.valueOf(n);
```

Substring boundaries form a half-open interval

- A substring starts at begin and stops before end.
- The difference end - begin gives its UTF-16 length.
- An empty substring is valid when begin equals end.

Example

```
"JAVA".substring(1,3) is "AV"
Included indexes: 1 and 2
Excluded index: 3
```

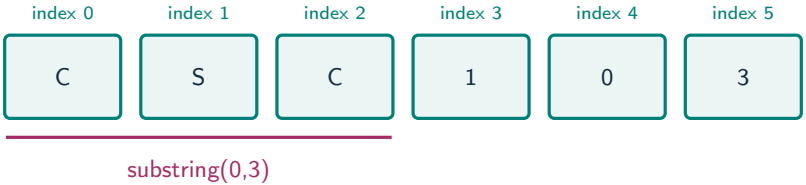
Search failure must be handled before slicing

- indexOf returns -1 when it finds no match.
- Passing -1 as a substring boundary is invalid.
- Validate the position before extracting the two parts of a structured value.

Example

```
For "ali@campus.edu": position=3
Username: substring(0,3)
Domain: substring(4)
```

Substring boundaries sit between characters



What to notice

substring(0,3) selects positions 0, 1 and 2. The end index 3 is excluded.

Key distinctions

Operation	Question answered	Example result
equals	Same text content?	"CS" equals "CS": true
==	Same object reference?	May differ for equal text
indexOf	Where is the first match?	"CS-103" finds "-" at 2
substring	Which interval of text?	substring(3) gives "103"

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
String code = "CS-103";
String prefix = code.substring(0, 2);
int number = Integer.parseInt(code.substring(3));
System.out.println(prefix);
System.out.println(number + 1);
```

First example: result

CS
104
The numeric substring is "103". Parsing enables arithmetic.

Substrings and searching
Numbers and text

Guided practice

For a nonempty email-like string with one @, extract the username and domain. Test ali@campus.edu.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Find the @ position and reject missing or empty parts.
- Extract text before @ as the username.
- Extract text after @ as the domain.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution13 {
  public static void main(String[] args) throws Exception {
    String email = "ali@campus.edu";
    int at = email.indexOf("@");
    if (at <= 0 || at == email.length() - 1) {
      System.out.println("Invalid format");
    } else {
      String user = email.substring(0, at);
      String domain = email.substring(at + 1);
      System.out.println(user);
      System.out.println(domain);
    }
  }
}
```

Execution trace

Part	Index interval	Result
Username	[0, 3)	ali
Separator	index 3	@
Domain	[4, 14)	campus.edu

Follow the changing state in execution order.

Solution result

ali
campus.edu

Why this result follows
Extract text after @ as the domain.

A common incorrect approach

String name = new String("Ali");
if (name == "Ali") { /* matched */ }

Example

```
Use name.equals("Ali") or "Ali".equals(name). == checks reference identity, not text content.
```

Boundary and failure checks

Check the stated input assumptions.
Read an item code shaped like BOOK-250. Extract the category and price, then print the price plus 10.

Find position with indexOf("@"). Use substring(0,pos) and substring(pos+1). Results: ali and campus.edu. Reject a missing @ before extracting.

Check your understanding

What is "JAVA".substring(1,3)? Does s.trim() change the String referred to by s?

Write a prediction and explain the rule that supports it.

Answer and explanation

"AV". No. Use s = s.trim() to keep the returned result.

Use name.equals("Ali") or "Ali".equals(name). == checks reference identity, not text content.

Compare cases and predict outcomes

Call on "CSC103"	Indexes used	Result
substring(0,3)	0, 1, 2	CSC
substring(3)	3, 4, 5	103
substring(6)	No characters	Empty string

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Given "lab:42", find the colon and convert the text after it to an int. State the assumptions needed for the conversion.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
String text = "lab:42";
int separator = text.indexOf(":");
String number = text.substring(separator + 1);
int value = Integer.parseInt(number);
System.out.println(value + 1);
```

- The colon is at index 3; substring(4) is "42".
- Parsing gives 42, so the output is 43.
- This example assumes a colon and a valid integer suffix; production code must validate them.

Lecture summary

- common String methods
- Compare text content and extract substrings
- Convert numeric text and numeric values

- Find the @ position and reject missing or empty parts.
- Extract text before @ as the username.
- Extract text after @ as the domain.

After-class practice

Read an item code shaped like BOOK-250. Extract the category and price, then print the price plus 10.

Syllabus reading
Deitel Ch 16
Next lecture: Built-in and instance methods